

SOFTWARE ENGINEERING (CS301) — LAB 3: COMPONENT MODELLING & ARCHITECTURAL SELECTION

ARCHITECTURE JUSTIFICATION: PROBLEM STATEMENT #46

REMOTE TEAM TIME-TRACKING & PROJECT APPROVER SYSTEM

Author: Sujan S Halanannavar

SRN: PES1UG24CS477

Institution: PES University

Date: October 2026

ARCHITECTURE SELECTION STATEMENT

“We chose 3-Tier Layered Architecture with a Client-Server deployment model for the Remote Team Time-Tracking & Project Approver System.”

1. ARCHITECTURAL CHOICE

We selected a 3-Tier Layered Architecture with a Client-Server deployment model. The system is divided into three main layers: the Client/Presentation Layer, the Application/Business Logic Layer, and the Data Layer.

The Client/Presentation Layer contains the Developer Web Client and Manager Dashboard, which provide the user interface for Remote Developers and Engineering Managers.

The Application/Business Logic Layer contains the Timesheet Manager, Validation Engine, Jira Integration Component, and Budget Analytics Component. This layer manages the main system rules, timesheet workflow, validation, approval, Jira integration, and budget calculations.

The Data Layer contains the Timesheet & Budget Database Component, which stores timesheet entries, approval and rejection records, audit information, and project budget data.

This structure separates user interaction, business logic, and data storage, making the system easier to understand, maintain, and manage.

2. TWO SPECIFIC SCENARIO-RELATED REASONS FOR SELECTION

Reason 1 — Centralized Management for Remote Teams:

Remote Developers can submit their timesheets from different locations, while Engineering Managers need to review and approve them through a central system. The Client-Server deployment model provides a single source of truth for timesheet information and approval status.

The system can therefore maintain a consistent workflow such as Draft → Pending Approval → Approved/Rejected. This reduces the possibility of inconsistent timesheet records and ensures that submitted information is managed centrally.

Reason 2 — Clear Separation of Jira Integration and Validation:

Developers need to record time against specific project tasks or Jira tickets. The dedicated Jira Integration Component handles communication with the Jira Project Management System, keeping Jira-related operations separate from the user interface.

The Validation Engine independently checks timesheet rules, including the requirement that a developer must not record more than 24 hours on a single calendar day. This separation makes the system easier to maintain because Jira integration and timesheet validation can be modified independently.

3. SECURITY ADVANTAGE

The system requires timesheet approval and rejection to be restricted to authenticated Engineering Managers.

The Application/Business Logic Layer can enforce role-based access control (RBAC) so that Developers can submit timesheets but cannot access manager-only approval functions.

Authentication and authorization can therefore be handled centrally before an approval or rejection operation is performed. Approval and rejection decisions can also be recorded in the database for auditing.

This supports the security requirement that only authenticated Engineering Managers can approve or reject submitted timesheets.

4. PERFORMANCE BENEFIT

The system requires timesheet approval status and project budget burn-rate information to be updated across dashboards within 500 ms.

The layered architecture separates the user interface from business logic and database operations. The Budget Analytics Component can handle burn-rate calculations, while the database provides the required timesheet and project information.

This separation allows dashboard requests to be processed efficiently and supports the required sub-500 ms dashboard update target.

COMPONENT AND INTERFACE SUMMARY

1. Developer Web Client Component

Layer: Client / Presentation Layer

Core Role:

Provides the interface for Developers to log time against project tasks or Jira tickets and submit completed timesheets.

Interface:

Requires: Timesheet REST API.

2. Manager Dashboard Component

Layer: Client / Presentation Layer

Core Role:

Allows Engineering Managers to review submitted timesheets, approve or reject them, and view project budget burn-rate information.

Interface:

Requires: Timesheet REST API.

3. Timesheet Manager Component

Layer: Application / Business Logic Layer

Core Role:

Manages the timesheet workflow, submission status, approval workflow, authentication, authorization, and audit operations.

Interfaces:

Provides: Timesheet REST API.

Requires: Database Persistence Interface, Validation Interface, and Jira Sync Interface.

4. Validation Engine Component

Layer: Application / Business Logic Layer

Core Role:

Validates submitted timesheets and enforces the maximum 24-hour daily limit and other timesheet validation rules.

Interface:

Provides: Validation Interface.

5. Jira Integration Component

Layer: Application / Business Logic Layer

Core Role:

Communicates with the Jira Project Management System and retrieves project tasks, tickets, and relevant worklog information.

Interface:

Provides: Jira Sync Interface.

6. Budget Analytics Component

Layer: Application / Business Logic Layer

Core Role:

Calculates and provides project budget burn-rate information for the Engineering Manager dashboard.

Interface:

Provides: Analytics Interface.

7. Timesheet & Budget Database Component

Layer: Data / Persistence Layer

Core Role:

Stores timesheet entries, approval and rejection records, audit information, project budgets, and related project data.

Interface:

Provides: Database Persistence Interface.

MAIN INTERFACES

1. Timesheet REST API

Purpose:

Connects the Client/Presentation Layer with the Timesheet Manager Component.

Example operations:

submitTimesheet()

reviewTimesheet()

approveTimesheet()

rejectTimesheet()

2. Jira Sync Interface

Purpose:

Connects the Timesheet Manager/Jira Integration Component with the external Jira Project Management System.

Example operations:

fetchTasks()

syncWorklogs()

3. Validation Interface

Purpose:

Connects the Timesheet Manager Component with the Validation Engine Component.

Example operations:

validateTimesheet()

check24hCap()

4. Analytics Interface

Purpose:

Connects the Manager Dashboard with the Budget Analytics Component.

Example operations:

getBudgetBurnRate()

getProjectCosts()

5. Database Persistence Interface

Purpose:

Connects the Application/Business Logic Layer with the Timesheet & Budget Database Component.

Example operations:

storeTimesheet()

logDecision()

queryMetrics()

ARCHITECTURE SUMMARY

Client / Presentation Layer



Application / Business Logic Layer



Data / Persistence Layer



EXTERNAL SYSTEMS AND ACTORS

Remote Developer:

Uses the Developer Web Client to log time and submit timesheets.

Engineering Manager:

Uses the Manager Dashboard to review, approve, reject timesheets, and view budget burn-rate information.

Jira Project Management System:

Provides project tasks, Jira tickets, and relevant worklog information through the Jira Integration Component.

FINAL ARCHITECTURE STATEMENT

The 3-Tier Layered Architecture with a Client-Server deployment model is appropriate for the Remote Team Time-Tracking & Project Approver System because it provides clear separation between user interaction, business rules, and data storage while supporting centralized timesheet management, secure approval workflows, Jira integration, and responsive budget and timesheet monitoring.